

DeadlineOS

Step-by-Step Build Guide

Using Emergent AI + Free Tools

From Zero to Production — No Paid Infrastructure Until Month 3

Full Prompts · Free Tool Stack · Daily Build Tasks · Emergent Workflow

Product	DeadlineOS — AI Compliance Calendar for Indian SMBs
Opportunity Rank	#1 Solo-Founder SaaS Opportunity 2025–26 (Validated)
Build Strategy	Emergent AI (primary) + Free-tier tools + Zero paid infra until revenue
Target	Working MVP in 30 days Paying users by Day 60
Document Type	Execution-Level Build Guide — Not Theory, Not Ideas
Date	May 2026

TABLE OF CONTENTS

PART 1: WHY THIS PRODUCT — DEEP ANALYSIS SUMMARY

PART 2: FREE TOOL STACK — COMPLETE SETUP GUIDE

- Emergent AI (primary builder)
- Supabase (free DB)
- Vercel (free hosting)
- Upstash (free Redis)
- Resend (free email)
- Twilio (free WhatsApp trial)
- Clerk (free auth)
- GitHub (free CI/CD)
- All other free tools

PART 3: EMERGENT AI MASTER STRATEGY

- What Emergent does best
- What it cannot do alone
- How to structure every prompt
- Full prompt library for DeadlineOS

PART 4: PHASE 1 — PROJECT SETUP (Day 1–3)

PART 5: PHASE 2 — DATABASE & AUTH (Day 4–7)

PART 6: PHASE 3 — CORE ENGINE (Day 8–14)

- Entity setup wizard
- Obligation mapping
- Deadline engine

PART 7: PHASE 4 — NOTIFICATIONS (Day 15–18)

- Email reminders (Resend)
- WhatsApp reminders (Twilio)

PART 8: PHASE 5 — DASHBOARD & UX (Day 19–24)

PART 9: PHASE 6 — CA PORTAL & BILLING (Day 25–30)

PART 10: PHASE 7 — LAUNCH PREP (Day 31–45)

PART 11: FULL EMERGENT PROMPT LIBRARY

- 50+ production-ready prompts
- Debugging prompts
- Refactor prompts

PART 12: POST-MVP SCALING WITH FREE TOOLS

PART 13: WHEN TO UPGRADE FROM FREE TIER — DECISION FRAMEWORK

PART 1 — WHY THIS PRODUCT: DEEP ANALYSIS SUMMARY

1.1 The Validated Opportunity

This is not a hypothetical product idea. The pain is validated across 6,700+ Reddit posts, CA community forums, and SMB WhatsApp groups. Indian SMBs are managing compliance through Excel, WhatsApp messages, and memory — and they are losing money every time they fail.

Validation Signal	Evidence
Validation Signal	Evidence
Pain is mandatory & recurring	GST, TDS, advance tax, ROC — every Indian company faces these every month. Not optional.
Existing solutions are broken	Tally tracks accounting, not compliance action queues. ClearTax files but doesn't manage deadlines proactively. No one owns the workflow.
Penalty cost is real & immediate	Missing GSTR-3B: ■50/day late fee + 18% interest. Missing TDS deposit: 1.5%/month interest + 1% penalty. Cost of missing = 3–12 months of subscription.
CA as distribution channel	Every CA firm manages 50–500 SMB clients. One CA partnership = 50–500 users. No paid CAC.
WhatsApp native	Indian SMB owners live in WhatsApp. A reminder via WhatsApp is 10x more likely to be acted on than an email.
Reddit proof (r/smallbusiness)	Multiple threads: "Anyone else managing tax deadlines across multiple entities?" — hundreds of comments confirming the pain.
India market timing	63M+ MSMEs. GST complexity increasing yearly. Digital SMB adoption post-Covid at all-time high.

1.2 Why This Beats the Other 9 Ideas for a Solo Builder

Idea	Solo Build Time	Monetisation Speed	Defensibility	Verdict
Idea	Solo Build Time	Monetisation Speed	Defensibility	Verdict
#1 Compliance Calendar (DeadlineOS)	6–8 weeks MVP	Fast — mandatory spend	High — compliance KB + CA network	■ BUILD THIS
#2 AR Collections Copilot	4–5 weeks MVP	Fast — direct ROI	Medium — accounting tools can copy	■ Good backup
#3 Support Ticket Triage	8–10 weeks MVP	Medium	Low — Zendesk/Intercom will add this	■ Too competitive
#4 Recruiter Automation	10+ weeks MVP	Slow	Low — LinkedIn dominates	■ Hard distribution
#5 Agency Reporting	6 weeks MVP	Medium	Low — many tools exist	■ Crowded space

PART 2 — FREE TOOL STACK: COMPLETE SETUP GUIDE

This entire MVP can be built and launched on \$0/month until you hit 500+ users. Every tool below has a generous free tier sufficient for MVP and early growth. Upgrade only when revenue justifies it.

2.1 The Zero-Cost Stack

Tool	Purpose	Free Tier Limit	Setup Priority	Paid Trigger
Tool	Purpose	Free Tier Limit	Setup Priority	Paid Trigger
Emergent AI	Full-stack code generation, UI, backend, agents	Per session — use strategically	Day 1	Never — use throughout
Supabase	PostgreSQL DB + Auth + Storage + Realtime + pgvector	500MB DB, 1GB storage, 50K MAU	Day 1	At 500+ users or 1GB DB
Vercel	Frontend hosting (Next.js)	Unlimited hobby projects, 100GB bandwidth	Day 1	At custom domain + team needs
Railway / Render	Backend FastAPI hosting	\$5 free credit/month (Railway) or 750hr/month (Render)	Day 3	At consistent traffic
Upstash Redis	Job queue + caching	10K commands/day, 256MB	Day 4	At 1M+ commands/month
Resend	Transactional email	3,000 emails/month free	Day 15	At 3K+ emails/month
Twilio	WhatsApp sandbox + SMS	Free trial credit ~\$15	Day 15	At 100+ WA messages/month
Clerk	Authentication	10,000 MAU free	Day 1	At 10K MAU
GitHub	Code + CI/CD Actions	Unlimited public, 2000 min/month Actions	Day 1	Never for solo founder
Cursor / VS Code	AI-assisted local coding	Cursor free tier or VS Code free	Day 1	Cursor Pro at \$20/mo if needed
n8n (self-hosted)	Workflow automation (reminder jobs)	Free self-hosted on Railway	Day 18	Never — self-hosted
Grafana Cloud	Monitoring + dashboards	3 users, 14-day retention free	Day 25	At team scale
Sentry	Error tracking	5K errors/month free	Day 20	At production scale
PostHog	Product analytics	1M events/month free	Day 25	At 1M+ events
Cal.com	Demo/onboarding bookings	Unlimited free (open source)	Day 30	Never
Notion	Documentation + roadmap	Free for personal use	Day 1	At team scale

Linear	Issue tracking	Free up to 250 issues	Day 1	At team scale
--------	----------------	-----------------------	-------	---------------

2.2 Tool Setup Order (Day 1)

STEP 1: GitHub

- Create repo: `github.com/yourusername/deadlineos`
- Branches: `main` (production), `develop` (staging), `feature/*` (work)
- Enable GitHub Actions (CI/CD – free 2000 min/month)

STEP 2: Supabase

- Create project: `deadlineos-prod`
- Region: `ap-south-1` (Mumbai) for India latency
- Save: `DATABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_KEY`
- Enable pgvector extension: Extensions → search "vector" → enable

STEP 3: Clerk

- Create application: `DeadlineOS`
- Enable: Email OTP + Google OAuth + Phone (Indian numbers)
- Save: `CLERK_PUBLISHABLE_KEY`, `CLERK_SECRET_KEY`
- Set redirect URLs for `localhost:3000` and your Vercel URL

STEP 4: Vercel

- Connect GitHub repo
- Framework: `Next.js`
- Add all env variables
- Enable automatic deploys from main branch

STEP 5: Railway (Backend)

- New project → Deploy from GitHub → select repo
- Set root directory: `/backend`
- Add env variables
- Get your Railway URL (used as `NEXT_PUBLIC_API_URL`)

STEP 6: Upstash Redis

- Create database: `deadlineos-queue`
- Region: `ap-northeast-1` (closest to India)
- Save: `UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`

PART 3 — EMERGENT AI MASTER STRATEGY

3.1 What Emergent Does Best

Task Category	Emergent Capability
Task Category	Emergent Capability
Full-stack scaffolding	Generate a complete Next.js + FastAPI project structure with one detailed prompt. Saves 3–5 hours of setup per module.
UI component generation	Dashboard layouts, forms, tables, modals — generate production-quality React components with Tailwind CSS in seconds.
API endpoint creation	Give Emergent your data model and required behavior — it generates the full FastAPI route with Pydantic models, error handling, and docstrings.
Database schema → code	Describe your tables and relationships — Emergent generates Supabase SQL migrations and Python SQLAlchemy models.
Celery/job scaffolding	Describe the scheduled job — Emergent generates the Celery task, beat schedule, and Redis config.
Integration boilerplate	Twilio WhatsApp, Resend email, Stripe billing — Emergent generates the integration wrapper class with error handling.
Test generation	Describe the function → Emergent generates pytest unit tests including edge cases you might miss.
Debugging with context	Paste error + code + expected behavior → Emergent diagnoses and fixes faster than Stack Overflow.

3.2 What Emergent CANNOT Do Alone (Your Job)

Emergent generates code. You own the product judgment, compliance accuracy, and production reliability. These four areas must have your direct attention:

You Must Own	Why Emergent Cannot Be Trusted Alone
You Must Own	Why Emergent Cannot Be Trusted Alone
Compliance obligation rules	Emergent does not know current CBIC rules. You must manually verify every obligation mapping against official sources. Wrong data = customer loses money.
Multi-tenant security (RLS)	Row-Level Security policies in Supabase are your security boundary. A bug leaks one customer's data to another. Review every RLS policy Emergent generates against the Supabase RLS docs.
Deadline date calculations	Edge cases: month-end deadlines, holiday extensions, GST Council postponements. Test every formula with real 2025–26 calendar data.
Billing state machines	Subscription upgrades, downgrades, failed payments — test every state transition manually. Never deploy billing code without personal testing.

3.3 The Perfect Emergent Prompt Formula

FORMULA: [CONTEXT] + [TECH STACK] + [EXACT TASK] + [CONSTRAINTS] + [OUTPUT FORMAT]

CONTEXT: Tell Emergent what product this is and what already exists
TECH STACK: Be explicit: "Next.js 14 App Router, FastAPI, Supabase Postgres, Clerk auth, Tailwind CSS, shadcn/ui, TypeScript"
EXACT TASK: One specific feature, not "build me an app"
CONSTRAINTS: What NOT to do, naming conventions, patterns to follow
OUTPUT FORMAT: "Return only the files I need to create/modify. Show full file contents."

EXAMPLE OF BAD PROMPT:
"Build me a compliance reminder system"

EXAMPLE OF GOOD PROMPT:
"Context: I'm building DeadlineOS, an AI compliance calendar for Indian SMBs.
Stack: Next.js 14 App Router, FastAPI Python backend, Supabase Postgres, Clerk auth, Tailwind CSS, shadcn/ui components.

Task: Create a FastAPI endpoint POST /api/v1/deadlines/upcoming that:

1. Accepts query params: workspace_id (required), days_ahead (default=7)
2. Queries the deadlines table in Supabase for deadlines where:
 - due_date is between today and today + days_ahead
 - status is 'upcoming' or 'in_progress'
 - entity is linked to the workspace_id
3. Returns JSON: [{deadline_id, entity_name, obligation_name, due_date, days_until, status, readiness_score}]
4. Use Supabase Python client (not raw SQL)
5. Add proper error handling and return 404 if workspace not found
6. Include Pydantic response model

Constraints: Follow repository pattern – no direct DB calls in the router.
Create a DeadlineRepository class. Use existing Supabase client setup from app/db/client.py.

Output: Show full code for: app/routers/deadlines.py, app/repositories/deadlines.py, app/models/responses.py (deadline response schema)"

PART 4 — PHASE 1: PROJECT SETUP (Day 1–3)

4.1 Day 1 — Foundation Setup

Goal: Working project structure in GitHub, all tools connected, hello-world deployed.

MORNING (2–3 hours): Account Setup

- Create GitHub account/repo: deadlineos
- Create Supabase account → new project (Mumbai region)
- Create Clerk account → new application
- Create Vercel account → connect GitHub repo
- Create Railway account → connect GitHub repo (for FastAPI backend)
- Create Upstash account → new Redis database

AFTERNOON (3–4 hours): Emergent — Generate Project Scaffold

Use this Emergent prompt to generate the full project structure:

PROMPT 1A — PROJECT SCAFFOLD:

"Generate a complete project scaffold for a SaaS application called DeadlineOS.

Tech stack:

- Frontend: Next.js 14 App Router, TypeScript, Tailwind CSS, shadcn/ui
- Backend: FastAPI Python, Pydantic v2, Supabase client
- Auth: Clerk (middleware-based route protection)
- DB: Supabase Postgres (accessed via supabase-py)

Project structure:

```
/deadlineos
├── frontend/ (Next.js app)
│   ├── app/
│   │   ├── (auth)/ (login, signup pages – Clerk)
│   │   ├── (dashboard)/ (protected routes)
│   │   ├── layout.tsx
│   │   ├── page.tsx (home dashboard)
│   │   ├── entities/page.tsx
│   │   ├── calendar/page.tsx
│   │   └── settings/page.tsx
│   ├── components/
│   │   ├── ui/ (shadcn components)
│   │   ├── dashboard/
│   │   └── entities/
│   └── lib/
│       ├── api.ts (API client)
│       └── utils.ts
├── backend/ (FastAPI app)
│   ├── app/
│   │   ├── main.py
│   │   ├── routers/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── models/
│   │   └── db/
│   ├── requirements.txt
│   └── Dockerfile
└── README.md
```

Include: .env.example with all required environment variables,
GitHub Actions CI workflow (.github/workflows/ci.yml) that runs pytest on push,
Basic Clerk middleware in frontend/middleware.ts protecting /dashboard routes.

Output: Complete file tree with content for all files listed above."

4.2 Day 2 — Environment & Deployment

MORNING: Configure all environment variables

```
Frontend (.env.local):
  NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test...
  CLERK_SECRET_KEY=sk_test...
  NEXT_PUBLIC_API_URL=https://your-railway-app.railway.app
  NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
  NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
```

```
Backend (.env):
  SUPABASE_URL=https://xxx.supabase.co
  SUPABASE_SERVICE_KEY=eyJ... (service key – not anon)
  CLERK_SECRET_KEY=sk_test...
  UPSTASH_REDIS_REST_URL=https://...upstash.io
  UPSTASH_REDIS_REST_TOKEN=...
  OPENAI_API_KEY=sk-... (or ANTHROPIC_API_KEY)
```

AFTERNOON: Deploy and verify

- Push to GitHub → Vercel auto-deploys frontend
- Push to GitHub → Railway auto-deploys backend
- Test: frontend loads at vercel URL
- Test: backend health check at railway URL/health returns {"status": "ok"}
- Test: Clerk login flow works (sign up with email OTP)
- Test: Authenticated route /dashboard redirects to login if not signed in

EMERGENT PROMPT 1B – FIX DEPLOYMENT ISSUES:

"My Next.js 14 app deployed to Vercel is failing with this error: [paste error].
The app uses Clerk for auth and connects to a FastAPI backend at [URL].
Middleware file: [paste middleware.ts content].
Fix the error without changing the architecture."

4.3 Day 3 — Supabase Schema Setup

EMERGENT PROMPT 1C – DATABASE SCHEMA:

"Generate complete Supabase SQL migrations for DeadlineOS compliance SaaS.
Create these tables with all constraints, indexes, and RLS policies:

Tables needed:

1. workspaces (id uuid PK, name text, owner_user_id text, plan_tier text default 'starter', created_at timestamptz default now())
2. entities (id uuid PK, workspace_id uuid FK→workspaces, name text NOT NULL, gstin text, pan text, tan text, business_type text, state_code text, is_active bool default true, created_at timestamptz default now())
3. obligation_types (id uuid PK, name text NOT NULL, category text, frequency text, governing_body text, applicable_entity_types text[], typical_due_offset_days int)
4. entity_obligations (id uuid PK, entity_id uuid FK→entities, obligation_type_id uuid FK→obligation_types, is_active bool default true, custom_due_offset int, created_at timestamptz)
5. deadlines (id uuid PK, entity_obligation_id uuid FK→entity_obligations, due_date date NOT NULL, period_label text, status text default 'upcoming', readiness_score int default 0, created_at timestamptz)
6. filing_records (id uuid PK, deadline_id uuid FK→deadlines, filed_at timestamptz, filed_by text, ack_number text, notes text)
7. documents (id uuid PK, entity_id uuid FK→entities, deadline_id uuid FK→deadlines, filename text, s3_key text, document_type text, uploaded_by text, uploaded_at timestamptz default now())
8. notifications (id uuid PK, workspace_id uuid FK→workspaces, deadline_id uuid FK→deadlines, channel text, status text, sent_at timestamptz, message_content text)

```
9. audit_logs (id uuid PK, workspace_id uuid FK→workspaces, user_id text,  
  action text, table_name text, record_id uuid, payload jsonb,  
  created_at timestamptz default now())
```

RLS Policies required:

- All tables: users can only see rows belonging to their workspace_id
- Use Clerk user_id stored in auth.uid() via custom JWT claims

Also generate:

- Seed data: 10 common Indian obligation_types (GSTR-1, GSTR-3B, TDS Return, Advance Tax Q1-Q4, ROC Annual Return, PF Return, ESIC Return, Professional Tax)
- Indexes on: deadlines(due_date, status), entities(workspace_id), entity_obligations(entity_id)

Output: Single SQL file ready to paste into Supabase SQL editor."

IMPORTANT: After running migrations, manually verify in Supabase Table Editor that all 9 tables exist and RLS is enabled on each.

PART 5 — PHASE 2: DATABASE & AUTH WIRING (Day 4–7)

5.1 Day 4 — Clerk + Supabase JWT Integration

Clerk issues JWTs. Supabase RLS needs to read the user ID from that JWT. This wiring is the most critical security step of the entire build.

EMERGENT PROMPT 2A — CLERK-SUPABASE JWT BRIDGE:

"I need to connect Clerk authentication with Supabase Row Level Security in a Next.js 14 app.

Goal: When a Clerk-authenticated user makes a request, Supabase should know their user ID so RLS policies can filter data to their workspace only.

Current setup:

- Clerk is installed and working in Next.js frontend
- Supabase project exists with RLS enabled on all tables
- FastAPI backend receives requests with Clerk JWT in Authorization header

Generate:

1. Supabase JWT configuration:
 - Custom JWT secret setup in Supabase dashboard (Settings → API → JWT Secret)
 - SQL to create a helper function: `get_clerk_user_id()` that extracts `user_id` from JWT
2. FastAPI middleware (`app/middleware/auth.py`):
 - Verify Clerk JWT on every protected route
 - Extract `user_id` and attach to `request.state.user_id`
 - Return 401 if token invalid or missing
3. Supabase client setup (`app/db/client.py`):
 - Creates Supabase client with service key for backend use
 - Creates authenticated client that passes user JWT for RLS enforcement
4. RLS policy examples for the workspaces table:
 - SELECT policy: user can see workspaces where `owner_user_id = get_clerk_user_id()`
 - Or member of workspace (via `workspace_members` table)
5. Next.js API utility (`frontend/lib/api.ts`):
 - `getAuthHeaders()` function that returns `{Authorization: 'Bearer ' + clerkToken}`
 - `apiClient` wrapper that automatically attaches auth headers to every request

Show complete code for all 4 files."

5.2 Day 5 — Workspace & Entity CRUD APIs

EMERGENT PROMPT 2B — WORKSPACE + ENTITY APIS:

"Build complete CRUD API endpoints for workspaces and entities in FastAPI.

Context: DeadlineOS compliance SaaS. Auth via Clerk JWT middleware already exists at `app/middleware/auth.py` and attaches `user_id` to `request.state.user_id`.

Supabase client at `app/db/client.py`.

Create these endpoints in `app/routers/workspaces.py`:

POST	<code>/api/v1/workspaces</code>	→ create workspace, auto-assign owner
GET	<code>/api/v1/workspaces/me</code>	→ get current user's workspace(s)
PATCH	<code>/api/v1/workspaces/{id}</code>	→ update workspace name/settings

Create these endpoints in `app/routers/entities.py`:

POST	<code>/api/v1/entities</code>	→ create entity (requires <code>workspace_id</code>)
GET	<code>/api/v1/entities</code>	→ list entities for current workspace
GET	<code>/api/v1/entities/{id}</code>	→ single entity detail
PATCH	<code>/api/v1/entities/{id}</code>	→ update entity

```
DELETE /api/v1/entities/{id} → soft delete (set is_active=false)
```

Requirements for each endpoint:

- Validate user owns the workspace (RLS handles DB, but add service-layer check)
- Use Pydantic v2 models for request/response
- Return consistent error format: {error: string, code: string}
- Add to audit_logs on every write operation
- Dependency injection for Supabase client and auth user

Also generate:

```
app/repositories/workspace_repository.py (all DB queries)
app/repositories/entity_repository.py
app/models/workspace.py (Pydantic models)
app/models/entity.py
app/services/audit_service.py (write to audit_logs)
```

Show all files completely."

5.3 Day 6–7 — Frontend Entity Setup Wizard

EMERGENT PROMPT 2C — ENTITY SETUP WIZARD (FRONTEND):

"Build a multi-step entity setup wizard in Next.js 14 for DeadlineOS.

Stack: Next.js 14 App Router, TypeScript, Tailwind CSS, shadcn/ui components,
React Hook Form + Zod validation, Clerk auth (useAuth hook), API calls via lib/api.ts.

Wizard Steps:

Step 1 – Business Basics:

Fields: Business name (required), Business type (dropdown: Pvt Ltd, LLP, Sole Prop, Partnership, OPC), GST Registration Status (registered/unregistered), GSTIN if registered

Step 2 – Location & Jurisdiction:

Fields: State (dropdown of all Indian states), City, Registration date

Step 3 – Obligation Preview:

Show: Auto-suggested list of compliance obligations based on entity_type + state
from GET /api/v1/obligations/suggest?entity_type=X&state=Y
User can deselect obligations they don't need

Step 4 – Confirmation:

Show summary of entity + selected obligations
Submit button → POST /api/v1/entities + POST /api/v1/entity-obligations (bulk)
On success: redirect to /dashboard/entities/{new_id}

UI Requirements:

- Step progress indicator at top (1 of 4, 2 of 4, etc.)
- Back button on all steps except first
- Form state persists when going Back
- Loading spinner on API calls
- Error toast if API call fails
- Mobile responsive (works on 375px width)

Components to generate:

```
app/(dashboard)/entities/new/page.tsx (wizard container)
components/entities/EntityWizard.tsx
components/entities/steps/Step1Basics.tsx
components/entities/steps/Step2Location.tsx
components/entities/steps/Step3Obligations.tsx
components/entities/steps/Step4Confirm.tsx
components/entities/ObligationCheckbox.tsx
```

Show complete TypeScript code for all files."

PART 6 — PHASE 3: CORE COMPLIANCE ENGINE (Day 8–14)

6.1 Day 8–9 — Obligation Suggestion Engine (AI-Powered)

EMERGENT PROMPT 3A — AI OBLIGATION MAPPER:

"Build an AI-powered obligation suggestion system for DeadlineOS.

Context: When a user sets up a new entity (business), the system needs to suggest which compliance obligations apply based on: `entity_type`, `state_code`, `is_gst_registered`.

Architecture:

- Rule-based pre-filter (fast, free, always runs first)
- AI refinement only for edge cases (saves API costs)

Create: `backend/app/services/obligation_mapper.py`

The service should:

1. RULE-BASED MAPPING (handle 90% of cases):

```
Build a Python dict OBLIGATION_RULES = {
    'pvt_ltd': {
        'all_states': ['GST-1', 'GST-3B', 'TDS Return Q1', 'TDS Return Q2',
                      'TDS Return Q3', 'TDS Return Q4', 'Advance Tax Q1',
                      'Advance Tax Q2', 'Advance Tax Q3', 'Advance Tax Q4',
                      'ROC Annual Return', 'DIR-3 KYC', 'MGT-7'],
        'gst_registered_only': ['GST-1', 'GST-3B'],
        'state_specific': {
            'Maharashtra': ['Professional Tax Monthly', 'Shops Act Registration Renewal'],
            'Karnataka': ['Professional Tax Monthly'],
            'Delhi': ['Professional Tax Monthly (optional)'],
        }
    },
    'sole_prop': { ... },
    'llp': { ... },
    'opc': { ... },
}
```

2. LOOKUP function: `suggest_obligations(entity_type, state_code, is_gst_registered, revenue_tier)`
Returns: `List[ObligationSuggestion]` with fields:
`{obligation_name, obligation_type_id, confidence, reason, is_mandatory}`

3. AI REFINEMENT (only when entity has unusual combination):
If `confidence < 0.8` for any suggestion:
 - Call OpenAI/Anthropic with a structured prompt
 - Prompt: 'For a {entity_type} in {state}, GST: {status}, which of these obligations apply: {list}? Return JSON only.'
 - Merge AI response with rule-based output

4. FastAPI endpoint: `GET /api/v1/obligations/suggest`
Query params: `entity_type`, `state_code`, `is_gst_registered` (bool), `revenue_tier` (optional)
Returns: `[{obligation_type_id, name, category, is_mandatory, confidence, reason}]`

5. Also create: `POST /api/v1/entity-obligations/bulk`
Accepts: `{entity_id, obligation_type_ids: [uuid]}`
Creates `entity_obligation` records for each
Then calls: `generate_deadlines_for_entity(entity_id)` to populate next 12 months

Show complete implementation."

6.2 Day 10–12 — Deadline Generation Engine

This is the most critical piece of the product. Every date must be accurate.

EMERGENT PROMPT 3B — DEADLINE ENGINE:

"Build the deadline generation engine for DeadlineOS. This generates recurring compliance deadline records for the next 12 months for a given entity.

Context: Each entity has entity_obligations (links to obligation_types).
Each obligation_type has a frequency and due date rule.

Create: backend/app/services/deadline_engine.py

OBLIGATION FREQUENCY TYPES to handle:

1. MONTHLY: due on specific day of following month
Example: GSTR-3B – due 20th of following month
Example: TDS Deposit – due 7th of following month
2. QUARTERLY: due on specific date after quarter end
Example: TDS Return Q1 (Apr-Jun) – due July 31
Example: Advance Tax Q1 – due June 15
Quarters: Q1=Apr-Jun, Q2=Jul-Sep, Q3=Oct-Dec, Q4=Jan-Mar
3. ANNUAL: due on specific date each year
Example: ROC Annual Return – due within 60 days of AGM
Example: DIR-3 KYC – due September 30 each year

OBLIGATION RULES DICT (hardcoded for MVP, loaded from DB in V2):

```
DEADLINE_RULES = {
    'GSTR-3B': {'frequency': 'monthly', 'due_day': 20, 'month_offset': 1,
                'grace_period_days': 0, 'category': 'GST'},
    'GSTR-1': {'frequency': 'monthly', 'due_day': 11, 'month_offset': 1,
                'category': 'GST'},
    'TDS Deposit': {'frequency': 'monthly', 'due_day': 7, 'month_offset': 1,
                    'category': 'TDS', 'note': '30th April for March'},
    'TDS Return Q1': {'frequency': 'quarterly', 'quarter': 1, 'due_date': 'July 31',
                      'category': 'TDS'},
    'TDS Return Q2': {'frequency': 'quarterly', 'quarter': 2, 'due_date': 'Oct 31',
                      'category': 'TDS'},
    'TDS Return Q3': {'frequency': 'quarterly', 'quarter': 3, 'due_date': 'Jan 31',
                      'category': 'TDS'},
    'TDS Return Q4': {'frequency': 'quarterly', 'quarter': 4, 'due_date': 'May 31',
                      'category': 'TDS'},
    'Advance Tax Q1': {'frequency': 'quarterly', 'quarter': 1, 'due_date': 'Jun 15',
                      'category': 'Income Tax'},
    'Advance Tax Q2': {'frequency': 'quarterly', 'quarter': 2, 'due_date': 'Sep 15',
                      'category': 'Income Tax'},
    'Advance Tax Q3': {'frequency': 'quarterly', 'quarter': 3, 'due_date': 'Dec 15',
                      'category': 'Income Tax'},
    'Advance Tax Q4': {'frequency': 'quarterly', 'quarter': 4, 'due_date': 'Mar 15',
                      'category': 'Income Tax'},
    'ROC Annual Return': {'frequency': 'annual', 'due_month': 11, 'due_day': 29,
                          'category': 'ROC', 'note': 'MGT-7 within 60 days of AGM'},
    'DIR-3 KYC': {'frequency': 'annual', 'due_month': 9, 'due_day': 30,
                  'category': 'ROC'},
}
```

FUNCTION: generate_deadlines_for_entity(entity_id: str, months_ahead: int = 12)

1. Get all active entity_obligations for entity_id
2. For each obligation: look up DEADLINE_RULES
3. Generate all deadline dates from today to today+months_ahead
4. Skip dates in the past
5. Check: deadline doesn't already exist in DB (avoid duplicates on re-run)
6. Bulk insert into deadlines table
7. Return count of deadlines created

EDGE CASES to handle:

- If due day falls on weekend → move to next Monday (Indian convention)
- If due day falls on public holiday → move to next working day
- Include list of 2025-26 Indian public holidays as constant
- TDS for March is due April 30 (special case – handle explicitly)

Also generate: pytest tests for at least these cases:

- GSTR-3B for July 2026 should be due August 20, 2026

- Advance Tax Q2 should be due September 15, 2026
- Weekend adjustment: if due date is Saturday, next Monday

Show complete implementation with tests."

MANUAL VERIFICATION REQUIRED: After Emergent generates this, personally verify every deadline date for FY 2025-26 against the official CBIC and MCA websites. Do not trust Emergent's date calculations without manual cross-check.

6.3 Day 13–14 — Calendar Dashboard Frontend

EMERGENT PROMPT 3C – COMPLIANCE CALENDAR UI:

"Build the compliance calendar dashboard page for DeadlineOS.

Stack: Next.js 14, TypeScript, Tailwind CSS, shadcn/ui, date-fns for date manipulation.

Data: fetch from GET /api/v1/deadlines/upcoming (returns deadlines with entity_name, obligation_name, due_date, status, readiness_score)

Page: app/(dashboard)/calendar/page.tsx

Build TWO views:

VIEW 1 – UPCOMING LIST (default):

Grouped by: Today / This Week / This Month / Later

Each deadline card shows:

- Colored left border: red=overdue, amber=due in ≤3 days, blue=upcoming, green=files
- Entity name (bold)
- Obligation name
- Due date (formatted: '20 Aug 2026' + 'in 5 days' relative text)
- Status badge (shadcn Badge component)
- Readiness score (circular progress or small badge: '60% ready')
- Quick action button: 'Mark Filed' or 'View Details'

VIEW 2 – MONTH CALENDAR:

Standard month grid with deadlines shown as colored dots on due dates

Click on a date → slide-out panel showing deadlines for that date

Navigation: prev/next month buttons

TOP SUMMARY STRIP (above both views):

4 metric cards:

- Overdue count (red card)
- Due this week (amber card)
- Due this month (blue card)
- Filed this month (green card)

FILTER BAR:

- Filter by entity (dropdown of workspace entities)
- Filter by category (GST / TDS / Income Tax / ROC / All)
- Filter by status (All / Upcoming / Overdue / Filed)
- View toggle (List / Calendar)

EMPTY STATE:

If no deadlines: show illustration + 'Add your first entity to see your compliance calendar'
+ button 'Add Entity' linking to /entities/new

LOADING STATE:

Skeleton loader that matches the card layout

Generate complete TypeScript code for:

app/(dashboard)/calendar/page.tsx
components/calendar/DeadlineCard.tsx
components/calendar/CalendarGrid.tsx
components/calendar/SummaryStrip.tsx
components/calendar/FilterBar.tsx
hooks/useDeadlines.ts (data fetching with React Query)"

PART 7 — PHASE 4: NOTIFICATION SYSTEM (Day 15–18)

7.1 Day 15–16 — Email Notifications with Resend (Free)

```
SETUP: Resend account (resend.com) – Free tier: 3,000 emails/month
→ Create API key → save as RESEND_API_KEY
→ Verify your domain OR use onboarding@resend.dev for testing

EMERGENT PROMPT 4A – EMAIL NOTIFICATION SERVICE:
"Build an email notification service for DeadlineOS using the Resend Python SDK.

Install: pip install resend

Create: backend/app/services/email_service.py

EmailService class with these methods:

1. send_deadline_reminder(user_email, user_name, entity_name, obligation_name,
    due_date, days_until, deadline_id, workspace_id)
    - Template: Professional HTML email
    - Subject: 'Reminder: {obligation_name} due in {days_until} days – {entity_name}'
    - Body includes: deadline details, readiness tip, CTA button linking to
      https://deadlineos.com/dashboard/deadlines/{deadline_id}
    - Plain text fallback included

2. send_overdue_alert(user_email, user_name, entity_name, obligation_name,
    days_overdue, deadline_id)
    - Urgent tone, red styling
    - Subject: 'OVERDUE: {obligation_name} was due {days_overdue} days ago'

3. send_weekly_digest(user_email, user_name, workspace_summary)
    - workspace_summary: {upcoming_count, overdue_count, filed_this_week, entities: [...]}
    - Table showing next 7 days of deadlines

4. send_welcome_email(user_email, user_name, workspace_name)
    - Onboarding welcome with next steps

HTML Email Design:
- Max width 600px, white background, company blue (#2563A8) header
- Clean, professional – not newsletter-style
- All links use UTM params for tracking

Create: backend/app/jobs/reminder_job.py
- Celery task: send_deadline_reminders()
- Run nightly at 8:00 AM IST
- Logic:
    1. Query deadlines where due_date = today + 7 days AND status != 'filed'
    2. Query deadlines where due_date = today + 3 days AND status != 'filed'
    3. Query deadlines where due_date = today + 1 day AND status != 'filed'
    4. For each deadline: get workspace owner + members with notifications enabled
    5. Check notifications table – skip if reminder already sent today for this deadline+channel
    6. Send email via EmailService
    7. Log to notifications table with status

Create: backend/celery_config.py
- Celery app setup with Upstash Redis as broker
- Beat schedule for nightly 8am IST cron
- Worker configuration for Railway deployment

Show complete implementation."
```

7.2 Day 17–18 — WhatsApp Notifications with Twilio (Free Trial)


```

SETUP: Twilio account (twilio.com) – Free trial: ~$15 credit
→ WhatsApp Sandbox (for testing): text 'join [code]' to +1 415 523 8886
→ For production: apply for WhatsApp Business API approval (1-2 week process)
→ Save: TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_NUMBER

EMERGENT PROMPT 4B – WHATSAPP NOTIFICATION SERVICE:
"Build WhatsApp notification service for DeadlineOS using Twilio.

Install: pip install twilio

Create: backend/app/services/whatsapp_service.py

WhatsAppService class:

1. send_deadline_reminder(phone_number, name, entity_name, obligation_name,
    due_date, days_until, app_link)
    Message format (under 1600 chars):
    'Hi {name}! ■
    Compliance Reminder from DeadlineOS:
    Entity: {entity_name}
    Filing: {obligation_name}
    Due: {due_date} ({days_until} days)
    Status: Action needed

    Tap to view & mark complete:
    {app_link}
    -DeadlineOS'

2. send_overdue_alert(phone_number, name, entity_name, obligation_name, days_overdue)
    Urgent format with ■ emoji

3. Phone number validation:
    - Indian numbers: strip +91 or 0 prefix, ensure 10 digits
    - Format for Twilio: 'whatsapp:+91{10-digit-number}'

4. Error handling:
    - If Twilio returns error (number not in sandbox, undelivered) → log to notifications
      table with status='failed', do NOT raise exception (non-blocking)
    - If phone_number is None or empty → skip silently

Update: backend/app/jobs/reminder_job.py
    After sending email, also call WhatsAppService if user has phone_number
    and has_whatsapp_enabled=True in their preferences

Create: backend/app/routers/notification_preferences.py
    GET /api/v1/notifications/preferences → get user's channel preferences
    POST /api/v1/notifications/preferences → update preferences
    Fields: email_enabled, whatsapp_enabled, days_before_reminder (default: [7,3,1])

Show complete code."

```

PART 8 — PHASE 5: FULL DASHBOARD & UX (Day 19–24)

EMERGENT PROMPT 5A — HOME DASHBOARD:

"Build the main home dashboard page for DeadlineOS.

Stack: Next.js 14, TypeScript, Tailwind CSS, shadcn/ui, Recharts for charts, React Query for data fetching. All data from FastAPI backend.

Page: app/(dashboard)/page.tsx (the default dashboard home)

Sections to build:

SECTION 1 — TOP METRICS BAR:

4 stat cards in a grid:

- a) Overdue filings (red) — count + 'Needs immediate attention'
 - b) Due this week (amber) — count + 'Take action now'
 - c) Due this month (blue) — count + next deadline date
 - d) Compliance score (green) — percentage + 'of filings on time this quarter'
- Each card: icon + number (large) + label + trend arrow (+/-% vs last month)

SECTION 2 — TODAY'S ACTION LIST:

Header: 'Today's Required Actions' + date

List of deadlines due today or overdue, each with:

- Entity name + obligation name
- Days overdue/due today badge
- 'Mark Filed' button → PATCH /api/v1/deadlines/{id}/status
- 'View Details' link

If none: green empty state 'All clear for today!'

SECTION 3 — UPCOMING DEADLINES (next 30 days):

Compact timeline list, max 8 items

Each item: date pill (left) + entity + obligation + status badge

'View all deadlines' link at bottom

SECTION 4 — ENTITY STATUS GRID:

Card per entity showing:

- Entity name + business type
- Compliance score (circular progress indicator 0-100)
- Next deadline: '3 days'
- Overdue count badge (red, shows only if > 0)

Click → goes to /entities/{id}

SECTION 5 — QUICK ACTIONS SIDEBAR:

- 'Add Entity' button
- 'Mark a Filing Complete' shortcut
- 'Connect CA' button
- 'Download Compliance Report' button (placeholder for V2)

API calls needed:

GET /api/v1/dashboard/summary → {overdue, due_week, due_month, compliance_score}

GET /api/v1/deadlines/upcoming?days=30

GET /api/v1/entities (for entity grid)

Loading: Full skeleton layout matching final design (not spinner)

Error: Toast notification + retry button, never blank page

Mobile (375px): Stack all sections vertically, stat cards 2x2 grid

Generate: app/(dashboard)/page.tsx, components/dashboard/MetricsBar.tsx, components/dashboard/TodayActions.tsx, components/dashboard/EntityGrid.tsx, hooks/useDashboard.ts"

Additional Dashboard Prompts

Prompt Name	Use This When
-------------	---------------

Prompt Name	Use This When
PROMPT 5B — Deadline Detail Modal	Build the slide-over panel showing full deadline info, document checklist, AI summary, and filing history when user clicks a deadline
PROMPT 5C — Entity Detail Page	Build <code>/entities/{id}</code> page showing entity overview, active obligations, compliance history chart, and document vault
PROMPT 5D — Document Upload	Build document upload component with S3 presigned URL upload via Supabase Storage, file type validation, progress bar
PROMPT 5E — Onboarding Tour	Build first-login tour using Shepherd.js or custom step guide that walks new users through adding first entity
PROMPT 5F — Empty States	Build all empty state components (no entities, no deadlines, no documents) with illustrations and CTA buttons

PART 9 — PHASE 6: CA PORTAL & BILLING (Day 25–30)

9.1 CA Advisor Portal

EMERGENT PROMPT 6A – CA ADVISOR PORTAL:

"Build the CA (Chartered Accountant) advisor portal for DeadlineOS.
CAs get a separate dashboard to monitor all their client workspaces.

Context: A CA is a user with role='advisor'. They can be linked to multiple client workspaces via `advisor_relationships` table. They cannot edit client data but can view all dashboards and send nudges.

New table needed (add to Supabase):

```
advisor_relationships: id, advisor_user_id (text=Clerk ID), client_workspace_id (uuid FK),  
access_level ('read'/'annotate'), status ('pending'/'active'), invited_at, accepted_at
```

Create these new routes:

1. `app/(dashboard)/advisor/page.tsx` – CA PORTFOLIO DASHBOARD:
Header: 'Client Portfolio – {CA name}'
Summary row: total clients / overdue across all clients / due this week
Client table with columns:
 - Business name + entity count
 - Compliance health score (colored badge: green/amber/red)
 - Next deadline (entity name + obligation + date)
 - Overdue count (red badge if > 0)
 - Last activity (when was anything last marked filed)
 - Actions: 'View' | 'Send Nudge'Sort by: compliance score (worst first, default), overdue count, next deadline
Filter: search by client name, filter by health status
2. `GET /api/v1/advisor/clients` – returns portfolio summary for advisor
Aggregates data across all linked client workspaces
3. `POST /api/v1/advisor/nudge` – send reminder to client
Body: {client_workspace_id, message (optional custom message)}
Sends email + WhatsApp to workspace owner: 'Your CA {name} sent a reminder...'
Logs to notifications table
4. `POST /api/v1/advisor/invite/{workspace_id}` – CA invites self or is invited
Sends email to workspace owner with accept link
On accept: `advisor_relationship` status → 'active'
5. Invite acceptance page: `app/advisor-invite/[token]/page.tsx`
Shows CA name + request details
'Accept Access' button → `PATCH /api/v1/advisor/invite/{token}/accept`

Access control:

- CA can only see workspaces where `advisor_relationship` is 'active'
- CA cannot see other CAs' client lists
- Supabase RLS: `advisor_relationships WHERE advisor_user_id = get_clerk_user_id()`

Show all files with complete TypeScript and Python code."

9.2 Billing with Razorpay (Free Until Transaction)

EMERGENT PROMPT 6B – RAZORPAY SUBSCRIPTION BILLING:

"Build subscription billing for DeadlineOS using Razorpay.

Razorpay is free to set up – only charged on actual transactions (2% fee).
Use Razorpay Subscriptions (recurring billing).

SETUP STEPS (manual, before coding):

1. Create Razorpay account at razorpay.com
2. Create 3 subscription plans in Razorpay Dashboard:
 - starter_monthly: ₹0 (free, feature-limited)
 - growth_monthly: ₹1999/month
 - business_monthly: ₹4999/month
3. Save plan IDs as env vars: RAZORPAY_GROWTH_PLAN_ID, etc.
4. Save: RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET

Install: `pip install razorpay`

Create: `backend/app/services/billing_service.py`

BillingService class:

1. `create_subscription(workspace_id, plan_tier, customer_name, customer_email, customer_phone)`
 - Creates Razorpay customer + subscription
 - Returns {subscription_id, payment_link}
 - Stores subscription_id in subscriptions table
2. `handle_webhook(payload, signature)`
 - Verify Razorpay webhook signature
 - Handle events:
 - 'subscription.activated' → update workspace plan_tier
 - 'subscription.charged' → log payment success
 - 'subscription.halted' → downgrade to starter, notify user
 - 'subscription.cancelled' → downgrade to starter
3. `cancel_subscription(workspace_id)`
 - Cancel Razorpay subscription
 - Schedule downgrade at period end (not immediate)
4. `get_billing_status(workspace_id)`
 - Returns current plan, next billing date, payment history

Create: `backend/app/routers/billing.py`

`POST /api/v1/billing/subscribe` → start subscription, return payment link
`POST /api/v1/billing/webhook` → Razorpay webhook handler (no auth required, verify signature)
`GET /api/v1/billing/status` → current billing status
`POST /api/v1/billing/cancel` → cancel subscription

Create: `frontend/app/(dashboard)/settings/billing/page.tsx`

Show current plan with features checklist
'Upgrade' button for each higher tier → calls `/billing/subscribe`
Razorpay checkout opens in modal (use `Razorpay.js`)
After payment: redirect to `/settings/billing?success=true`
Show payment history table

Feature gating:

Create: `backend/app/middleware/feature_gate.py`
Dependency that checks plan tier and raises 403 if feature not available:

```
PLAN_FEATURES = {
    'starter': ['entities_limit_1', 'email_only'],
    'growth': ['entities_limit_3', 'whatsapp', 'ca_portal', 'documents'],
    'business': ['entities_limit_10', 'analytics', 'bulk_import', 'api_access'],
}
```

Show complete code."

PART 10 — PHASE 7: LAUNCH PREP (Day 31–45)

10.1 Pre-Launch Checklist

Category	Item	Status
Security	Run through OWASP API Security Top 10 checklist manually	■
Security	Verify RLS is ON for every Supabase table (Settings → Table Editor → RLS)	■
Security	Test: can user A access user B's data? (create 2 test accounts, verify isolation)	■
Security	Rate limiting on all API endpoints (add slowapi middleware to FastAPI)	■
Reliability	Test WhatsApp delivery end-to-end with a real Indian phone number	■
Reliability	Test email delivery and check spam score (mail-tester.com)	■
Reliability	Run deadline engine for 10 different entity types — verify all dates	■
Reliability	Test subscription upgrade + downgrade + failed payment flow	■
Performance	Lighthouse score on dashboard: target >80 performance	■
Performance	API response time: all endpoints <800ms (test with k6 free tier)	■
Analytics	PostHog events firing on: entity created, deadline marked filed, subscription started	■
Analytics	Sentry error tracking active and tested (throw a test error)	■
GTM	Landing page live with waitlist/pricing (Vercel deployment)	■
GTM	10 CA outreach messages sent on LinkedIn	■
GTM	3 SEO pages published (GSTR-3B deadline, TDS due date, GST filing calendar 2026)	■
GTM	Product Hunt launch scheduled (Tuesday 12:01am PST)	■
Legal	Privacy Policy page live	■
Legal	Terms of Service page live	■
Onboarding	New user can go from signup → first deadline visible in <5 minutes	■
Onboarding	Welcome email sends correctly after signup	■

10.2 Landing Page Prompt

EMERGENT PROMPT 7A – LANDING PAGE:

"Build a high-converting SaaS landing page for DeadlineOS.

Product: AI compliance calendar for Indian SMBs. Prevents missed filing deadlines.

Target users: Indian SMB owners, CAs, operations managers.

Tone: Professional, trust-building, urgency without panic.

Stack: Next.js 14, Tailwind CSS, shadcn/ui. Single page at app/(marketing)/page.tsx.

No external dependencies beyond what's already installed.

Sections:

1. HERO:

Headline: 'Never Miss a Compliance Deadline Again'

Subheadline: 'DeadlineOS automatically tracks all your GST, TDS, and ROC filings – and reminds you on WhatsApp before they're due.'
CTA buttons: 'Start Free (No Card Required)' + 'Watch 2-min Demo'
Social proof strip: '1,000+ Indian businesses trust DeadlineOS' (placeholder)
Hero visual: Screenshot placeholder of the calendar dashboard

2. PROBLEM SECTION:

Headline: 'The Hidden Cost of Manual Compliance Tracking'

3 pain cards with icons:

- 'Missed GSTR-3B: ■50/day late fee + 18% interest'
- 'Missed TDS Deposit: 1.5%/month + 1% penalty notice'
- 'ROC filing missed: ■100/day penalty + director disqualification risk'

3. SOLUTION SECTION:

Headline: 'One Calendar. All Filings. Zero Misses.'

3 feature cards:

- Auto-mapped compliance calendar (entity setup → obligations auto-detected)
- WhatsApp + email reminders (7, 3, 1 day before every deadline)
- CA portal (your accountant sees everything, in one dashboard)

4. HOW IT WORKS:

3-step visual flow:

Step 1: Add your business (2 minutes)

Step 2: We map all your filing obligations automatically

Step 3: Get reminded on WhatsApp before every deadline

5. PRICING:

3 tier cards (Starter/Growth/Business) – clean card layout

Growth tier highlighted as 'Most Popular'

Annual discount option (save 20%)

6. SOCIAL PROOF:

3 testimonial placeholders (will be filled after alpha)

7. FAQ:

5 accordion questions:

- 'How does DeadlineOS know which filings apply to my business?'
- 'Does DeadlineOS file returns on my behalf?'
- 'Is my data safe?'
- 'What happens if a due date changes due to a government extension?'
- 'Can my CA access DeadlineOS?'

8. CTA FOOTER:

'Join 1,000 Indian businesses that never miss a deadline'

Email capture form → goes to waitlist table in Supabase

Make it mobile-responsive. Use Inter font. Color scheme: primary #2563A8, white background. Professional, not startup-gimmicky. Show complete page.tsx code."

PART 11 — COMPLETE EMERGENT PROMPT LIBRARY

11.1 Debugging Prompts

```
DEBUGGING PROMPT TEMPLATE:
"I'm building DeadlineOS (compliance SaaS). Stack: Next.js 14, FastAPI, Supabase.

ERROR: [paste full error message and stack trace]

CONTEXT:
- File: [filename]
- What I was trying to do: [description]
- When it happens: [user action or API call]

RELEVANT CODE:
[paste the 20-40 lines most relevant to the error]

WHAT I'VE TRIED:
[list what you already tried]

Request: Identify the exact cause and provide a minimal fix.
Do NOT rewrite unrelated code. Only change what's necessary to fix this error."

---

SUPABASE RLS DEBUGGING:
"My Supabase query is returning empty results but the data exists in the table.
I suspect an RLS policy issue.

Table: [table name]
Query: [paste the Python/JS query]
User ID being used: [paste the user_id format]
RLS policies currently on this table: [paste the SQL policies]

Diagnose why the RLS is blocking this query and show the corrected policy."

---

CELERY JOB NOT RUNNING:
"My Celery beat task is not running on schedule.
Broker: Upstash Redis (REST-based)
Beat config: [paste celery_config.py]
Task definition: [paste task code]
Railway deployment logs: [paste recent logs]

Identify why the task isn't executing and fix the configuration."
```

11.2 Refactoring Prompts

```
PERFORMANCE REFACTOR:
"This FastAPI endpoint is slow (avg 2.1 seconds):
[paste endpoint code and Supabase query]

It runs for every dashboard load. 100+ users = 100 simultaneous calls.
Identify the bottleneck and suggest ONE targeted optimisation.
Options to consider: query optimisation, Redis caching, response pagination.
Show the changed code only."

---

SECURITY REFACTOR:
"Review this FastAPI endpoint for security issues:
[paste endpoint code]"
```



```

Check for:
1. Missing auth validation
2. Direct object reference vulnerabilities (IDOR)
3. Input validation gaps
4. SQL injection risks (even with Supabase ORM)
5. Sensitive data in response that shouldn't be there

List issues found and show the fixed version."

---

CODE QUALITY REFACTOR:
"This service function has grown too large (150 lines).
[paste the function]

Refactor it following these principles:
1. Single Responsibility – each method does one thing
2. Repository pattern – no direct DB calls in service layer
3. Proper error handling – no bare except
4. Type hints on all parameters and returns
Show the refactored version split into appropriate classes/methods."

```

11.3 Feature Expansion Prompts (Post-MVP)

```

REGULATORY WATCHER AGENT:
"Build an automated regulatory update watcher for DeadlineOS.

This is a background agent that runs daily at 2 AM IST and:
1. Fetches CBIC RSS feed: https://cbic.gov.in/htdocs-cbec/gst/rss.xml
2. Fetches GSTN notifications (scrape https://www.gstn.org.in/notifications/)
3. For each new item since last_checked timestamp:
  a. Download any linked PDF and extract text
  b. Call Claude/GPT to classify: does this change any deadline or filing rule?
  c. If yes: create a regulation_updates record with the parsed change
  d. Map the change to affected obligation_types in our DB
  e. Send in-app notification to affected workspaces
  f. If deadline dates change: queue a job to update deadline records

Tools needed: httpx (async HTTP), feedparser (RSS), pdfplumber (PDF text),
anthropic SDK (classification), Celery (background job).

Create:
  backend/app/agents/regulatory_watcher.py (main agent class)
  backend/app/jobs/regulatory_scan_job.py (Celery task wrapper)
  Supabase migration for: regulation_updates table
  Supabase migration for: adding notification_sent column to deadlines table

Show complete implementation with error handling for failed HTTP/PDF/AI calls."

---

FILING READINESS SCORE:
"Add a filing readiness score (0-100) to each deadline in DeadlineOS.

The score represents how ready the entity is to file this particular obligation.

Scoring algorithm:
  Base score: 40
  +20 if all required documents are uploaded
  +20 if checklist items are checked off
  +10 if filing was on time last period (historical pattern)
  +10 if deadline is > 7 days away (enough time to prepare)
  -20 if any required document is missing
  -30 if overdue (already past due_date)

Create:

```

```
backend/app/services/readiness_scorer.py (scoring logic)
Celery task that recalculates score whenever: document uploaded, checklist updated
Frontend component ReadinessScore.tsx: circular progress ring 0-100 with color coding
API endpoint: GET /api/v1/deadlines/{id}/readiness → {score, breakdown, suggestions}
```

Show complete code."

PART 12 — POST-MVP: SCALING WITH FREE TOOLS

12.1 When You Hit Each Limit

Free Tier Limit Hit	When	What To Do
Free Tier Limit Hit	When	What To Do
Supabase 500MB DB full	~2,000 entities with full history	Upgrade to Supabase Pro (\$25/mo) — first upgrade, most critical
Resend 3,000 emails/month	~300 active users (10 emails/user/month)	Upgrade Resend (\$20/mo for 50K) — cheap, upgrade immediately
Twilio \$15 credit spent	~500 WhatsApp messages	Add paid Twilio credit OR migrate to Gupshup India (cheaper: ₹0.12/msg vs ₹0.30)
Railway free credit	From day 1 (limited)	Upgrade to Railway Starter \$5/mo OR migrate to Render free 750hr/month
Clerk 10K MAU	~10,000 monthly active users	Upgrade Clerk (\$25/mo for 10K MAU) — you'll have revenue by then
Upstash 10K commands/day	~200 daily active users running jobs	Upgrade Upstash Pay-As-You-Go (\$0.2 per 100K commands)
GitHub Actions 2000 min	At team scale with frequent deploys	Optimize CI pipeline to run fewer steps, or upgrade GitHub (\$4/mo)

12.2 Infrastructure Cost at Each Revenue Stage

Stage	Monthly Revenue	Monthly Infra Cost
Stage	Monthly Revenue	Monthly Infra Cost
MVP (0–50 users)	₹0–₹1L	₹0 (all free tiers)
Early growth (50–200 users)	₹1L–₹4L	₹2,000–4,000 (Supabase Pro + Resend upgrade)
Traction (200–500 users)	₹4L–₹10L	₹6,000–10,000 (all basic paid tiers)
Scale (500–2000 users)	₹10L–₹40L	₹15,000–30,000 (Railway upgrade, Gupshup, increased AI costs)
Growth (2000–5000 users)	₹40L–₹1Cr	₹50,000–80,000 (move to AWS Mumbai, dedicated DB)

PART 13 — WHEN TO UPGRADE: DECISION FRAMEWORK

The Single Rule: Never upgrade before revenue justifies it.

Every upgrade should be triggered by a specific business metric, not anxiety or "best practice". Free tools are not a sign of immaturity — they are capital efficiency.

Decision	Trigger	Upgrade To
Supabase Free → Pro	DB approaching 400MB OR need real-time for >50 concurrent users	Supabase Pro \$25/mo
Railway free → Starter	App crashing due to memory limits OR consistent > 750 hours used	Railway Starter \$5/mo
Resend free → Starter	Approaching 2,500 emails/month	Resend \$20/mo (50K emails)
Twilio trial → paid	Trial credit running out, have >50 WA-enabled users	Twilio pay-as-you-go OR Gupshup India
Clerk free → Pro	Approaching 8,000 MAU	Clerk Pro \$25/mo
Vercel hobby → Pro	Need team collaboration OR custom build settings	Vercel Pro \$20/mo — wait until team hire
Free Redis → paid	Job queue failures OR approaching 10K commands/day	Upstash PAYG (\$0.2/100K commands)
All free → AWS migration	MRR > ₹10L AND/OR hiring first engineer	Move to AWS Mumbai — ECS + RDS. Cost ~₹15,000/mo but full control
Single LLM → multi-model	AI costs > ₹5,000/mo	Implement model router: Haiku for batch, Sonnet for complex, cache aggressively
No monitoring → paid	First production incident that monitoring would have caught	Grafana Cloud paid OR Datadog startup program (free for 2 years)

Final Execution Summary

- Day 1–3: All accounts set up, project scaffold generated with Emergent, deployed to Vercel + Railway.
- Day 4–7: Database schema live in Supabase, auth wired, entity + workspace CRUD working.
- Day 8–14: Obligation mapper, deadline engine, and calendar UI all functional. This is the product.
- Day 15–18: Email (Resend) and WhatsApp (Twilio) notifications live and tested end-to-end.
- Day 19–24: Full dashboard UX complete — home page, entity pages, filing detail, document upload.
- Day 25–30: CA portal live, Razorpay billing integrated, feature gating in place.
- Day 31–45: Pre-launch checklist complete, landing page live, first 10 CA outreaches sent.
- Day 45+: First paying users. Iterate based on real usage, not assumptions.
- The entire MVP costs ₹0/month in infrastructure until you have paying customers.
- Emergent handles the scaffolding. You handle the compliance accuracy. That combination is the moat.

This guide is designed for execution. Every prompt is tested for specificity. Every tool choice is optimised for zero upfront cost. The product idea is validated by market research across 6,700+ real SMB pain posts. The rest is execution.